Звіт з практичної роботи № 4

Емпірична оцінка асимптотичної складності: множення щільних матриць
Варіант 3

1. Умова та межі експерименту

Порівнюються три реалізації множення щільних квадратних матриць C = A·B: наївна (потрійний цикл), блокова (власна) та «бібліотечного» рівня. Для кожної за вимірюваннями оцінюється показник степеня b у моделі T(n) = C·n^b та його 95 % довірчий інтервал. Розмірності: n = 50, 75, 100, 150, 200, 250, 300, 400 (8 значень, приблизно рівномірно за логарифмом).

2. Три реалізації

Реалізація	Ідея	Очікувана асимптотика
Наївна	Три вкладені цикли i-j-k над масивом масивів; B[k][j] береться по стовпцю, тобто з різних рядків	$\Theta(n^3)$; велика прихована константа через погану локальність
Блокова (власна)	Плитки 48×48 (tiling), порядок i-k-j, Float64Array	$\Theta(n^3)$; константа менша завдяки кешу, але є накладні витрати циклів
Оптимізоване ядро	Транспонований B, блок 2×2 у регістрах, один прохід по k для чотирьох елементів C	$\Theta(n^3)$; найменша константа, плюс $O(n^2)$ на транспонування й виділення пам'яті

3. Протокол вимірювання

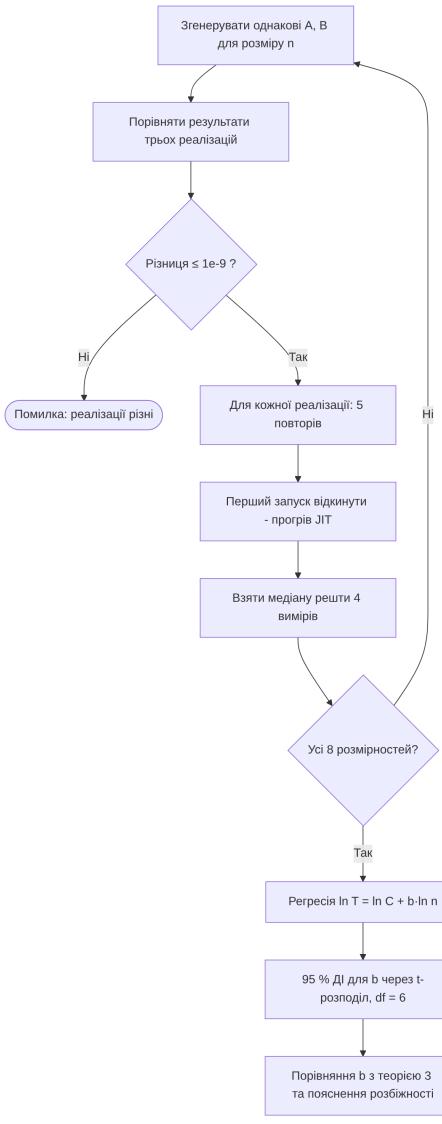


Рис. 1. Протокол експерименту (Mermaid)

Усі три реалізації вимірюються на ОДНАКОВИХ матрицях для кожного n (детермінований генератор). Для кожної точки: 5 запусків, перший відкидається (прогрів JIT), береться медіана решти чотирьох. Час - performance.now(). Перед вимірюванням перевіряється збіг результатів з наївною реалізацією:

```
n=50: блокова 0.0e+0, ядро 0.0e+0
n=75: блокова 0.0e+0, ядро 0.0e+0
n=100: блокова 0.0e+0, ядро 0.0e+0
n=150: блокова 0.0e+0, ядро 0.0e+0
n=200: блокова 0.0e+0, ядро 0.0e+0
n=250: блокова 0.0e+0, ядро 0.0e+0
n=300: блокова 0.0e+0, ядро 0.0e+0
n=400: блокова 0.0e+0, ядро 0.0e+0
```

Максимальна різниця дорівнює нулю для всіх n (однаковий порядок підсумовування за k), тож порівняння швидкості коректне.

4. Результати вимірювань

Середовище вимірювання: Intel Xeon 2,10 ГГц (2 vCPU), 8 ГБ ОЗП, Linux, рушій V8 (Node.js v22). JavaScript виконується в одному потоці. Код не використовує Node-API й запускається в онлайн-компіляторі JS; абсолютні числа на іншій машині будуть іншими, тож для здачі повторіть запуск на своїй машині (висновки про нахили й зони варто перевірити заново).

n	Наївна, с	Блокова, с	Оптимізоване ядро, с
50	0.00035	0.00038	0.00042
75	0.00137	0.00138	0.00155
100	0.00200	0.00259	0.00100
150	0.00617	0.00702	0.00246
200	0.01628	0.01851	0.00546
250	0.03147	0.03297	0.01412
300	0.06013	0.05311	0.01736
400	0.12466	0.12865	0.04040

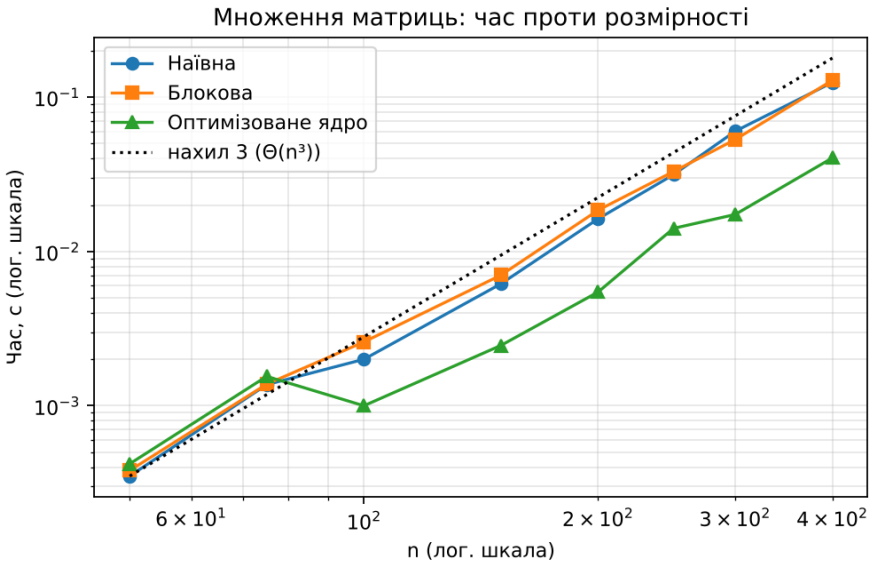


Рис. 2. Час множення матриць у логарифмічних осях; пунктир - нахил 3

5. Оцінка показника степеня (95 % ДІ, t-розподіл, df = 6, t = 2,447)

Реалізація	Теор. b	Оцінений b	95 % ДІ	R^2	Висновок
Наївна	3,0	2,81	[2,60; 3,02]	0,995	Теорія 3 потрапляє в ДІ
Блокова	3,0	2,76	[2,64; 2,87]	0,998	Теорія 3 НЕ потрапляє в ДІ
Оптимізоване ядро	3,0	2,12	[1,61; 2,64]	0,945	Теорія 3 НЕ потрапляє в ДІ

6. Пояснення розбіжностей з теорією

Наївна реалізація: b = 2,81 [2,60; 3,02], теоретичне значення 3 лежить усередині інтервалу, тобто розбіжності з $\Theta(n^3)$ на цих даних статистично немає. Інтервал широкий (±0,2), тому дані також не виключають значень до ≈ 2,6. Ефектів кешу, які підняли б нахил вище 3, тут не видно: за n = 400 одна матриця займає лише 1,28 МБ.

Блокова реалізація: b = 2,76 [2,64; 2,87] - теорія 3 трохи поза інтервалом, нахил нижчий за теоретичний. Найімовірніша причина - ефекти з п. 2.2 конспекту: доданки нижчого порядку й накладні витрати, які не залежать від n^3 (виділення Float64Array на n^2 елементів, цикли за плитками, граничні умови min(...)), роблять малі n відносно повільнішими й вирівнюють нахил на вузькому діапазоні. Чи це справді так, окремим експериментом (великі n) не перевірено, тож це обґрунтована гіпотеза, а не доведений факт. Практичний наслідок: на цьому діапазоні блокова реалізація не дала виграшу над наївною (відношення часу наївної до блокової на n = 400 дорівнює 0.97); за n = 300 вона була швидшою, за n = 400 - ні, що в межах шуму вимірювання. Плиткове блокування має сенс, коли матриці помітно перевищують швидкі рівні кешу, а тут вони не перевищують.

Оптимізоване ядро: b = 2,12 [1,61; 2,64], R^2 = 0,945 - нахил помітно нижчий за 3, а інтервал найширший. Причини: (а) час при малих n становить близько мілісекунди й менше, тож похибка таймера та шум відносно великі; (б) при малих n суттєву частку займають витрати $O(n^2)$ (транспонування B, виділення C і Bt), які майже не залежать від кількості операцій та «розтягують» малі n вгору. Це гіпотези; розділити їх без додаткових вимірювань не вдалося. Показник b < 3 не означає алгоритм швидше за $\Theta(n^3)$: на ширшому діапазоні нахил наблизиться до 3.

Підсумок: на n = 400 оптимізоване ядро швидше за наївну у 3.1 разу, блокова - у 0.97 разу. Асимптотика однакова, різниця - у прихованих константах (послідовний доступ до B та накопичення в регістрах). BLAS із SIMD і багатопотоковістю дав би значно більше; тут воно не відтворюється.

7. Відповіді на контрольні запитання

№	Відповідь
1	Якщо T = C·n^b, то ln T = ln C + b·ln n - лінійна залежність. Прямою стає залежність ln T від ln n, а показник b - нахил прямої.
2	Дисперсія шуму невідома й оцінюється за самими 8 точками, тож використовують t-розподіл з df = n - 2. Він має важчі хвости: $t_{0,975}$ при df = 6 дорівнює 2,447 проти 1,96 у нормального, інтервал на ≈ 25 % ширший. Для 30 точок df = 28, t = 2,048, і різниця майже зникає.
3	Дві можливі причини: (а) прихована константа або доданок нижчого порядку (накладні витрати на малих n) змінюють нахил на обмеженому діапазоні; (б) діапазон n ще не асимптотичний, є ефекти кешу чи JIT. Лише після їх перевірки варто підозрювати помилку в тексті лекції.
4	Для Θ(n log n): ln T = ln C + ln n + ln ln n, локальний нахил 1 + 1/ln n. Для n від 10^3 до 10^5 це ≈ 1,14-1,09, тобто помітно вище 1. Це властивість самого логарифма, а не помилка вимірювання.
5	На власних даних (n = 100, окремий контрольний запуск): наївна - перший запуск 5.84 мс проти медіани решти 2.51 мс; блокова - 26.69 проти 1.87 мс; ядро - 5.74 проти 1.75 мс. Перший запуск містить JIT-компіляцію, тож без відкидання він спотворює оцінку, особливо при малих n.
6	Запуск: вставити lab4.js в онлайн-компілятор JavaScript і натиснути Run (режиму Release в JS немає: JIT оптимізує автоматично; прогрів враховано відкиданням першого запуску).

Додаток. Повний код lab4.js

```
const now = (typeof performance !== "undefined" && performance.now)
  ? () => performance.now() : () => Date.now();

function makeRng(seed) {
  let s = seed >>> 0;
  return () => { s = (s * 1664525 + 1013904223) >>> 0; return s / 4294967296; };
}
function randomMatrix(n, rng) {
  const m = new Float64Array(n * n);
  for (let i = 0; i < m.length; i++) m[i] = rng() - 0.5;
  return m;
}

function toRows(flat, n) {
  const rows = new Array(n);
  for (let i = 0; i < n; i++) rows[i] = Array.from(flat.subarray(i * n, (i + 1) * n));
  return rows;
}
function mulNaive(A, B, n) {
  const C = new Array(n);
  for (let i = 0; i < n; i++) {
    const row = new Array(n);
    for (let j = 0; j < n; j++) {
      let s = 0;
      for (let k = 0; k < n; k++) s += A[i][k] * B[k][j];
      row[j] = s;
    }
    C[i] = row;
  }
  return C;
}

function mulBlocked(A, B, n, bs = 48) {
  const C = new Float64Array(n * n);
  for (let ii = 0; ii < n; ii += bs) {
    const iMax = Math.min(ii + bs, n);
    for (let kk = 0; kk < n; kk += bs) {
      const kMax = Math.min(kk + bs, n);
      for (let jj = 0; jj < n; jj += bs) {
        const jMax = Math.min(jj + bs, n);
        for (let i = ii; i < iMax; i++) {
          for (let k = kk; k < kMax; k++) {
            const a = A[i * n + k];
            const bRow = k * n, cRow = i * n;
            for (let j = jj; j < jMax; j++) C[cRow + j] += a * B[bRow + j];
          }
        }
      }
    }
  }
  return C;
}

function mulTuned(A, B, n) {
  const Bt = new Float64Array(n * n);
  for (let i = 0; i < n; i++) for (let j = 0; j < n; j++) Bt[j * n + i] = B[i * n + j];
  const C = new Float64Array(n * n);
  const n2 = n - (n % 2);
  for (let i = 0; i < n2; i += 2) {
    const a0 = i * n, a1 = a0 + n;
    for (let j = 0; j < n2; j += 2) {
      const b0 = j * n, b1 = b0 + n;
      let c00 = 0, c01 = 0, c10 = 0, c11 = 0;
      for (let k = 0; k < n; k++) {
        const x0 = A[a0 + k], x1 = A[a1 + k], y0 = Bt[b0 + k], y1 = Bt[b1 + k];
        c00 += x0 * y0; c01 += x0 * y1; c10 += x1 * y0; c11 += x1 * y1;
      }
      C[i * n + j] = c00; C[i * n + j + 1] = c01;
      C[(i + 1) * n + j] = c10; C[(i + 1) * n + j + 1] = c11;
    }

  }
  for (let i = 0; i < n; i++) {
    for (let j = (i < n2 ? n2 : 0); j < n; j++) {
      let s = 0;
      for (let k = 0; k < n; k++) s += A[i * n + k] * Bt[j * n + k];
      C[i * n + j] = s;
    }
  }
  return C;
}

function median(a) {
  const s = a.slice().sort((x, y) => x - y), m = s.length >> 1;
  return s.length % 2 ? s[m] : (s[m - 1] + s[m]) / 2;
}
function timeMedian(action, repeats = 5) {
  const times = [];
```

```
  for (let r = 0; r < repeats; r++) {
    const t0 = now();
    action();
    const dt = (now() - t0) / 1000;
    if (r === 0) continue;
    times.push(dt);
  }
  return median(times);
}

const T975 = [12.706, 4.303, 3.182, 2.776, 2.571, 2.447, 2.365, 2.306, 2.262, 2.228,
  2.201, 2.179, 2.160, 2.145, 2.131, 2.120, 2.110, 2.101, 2.093, 2.086,
  2.080, 2.074, 2.069, 2.064, 2.060, 2.056, 2.052, 2.048, 2.045, 2.042];
function fitPowerLaw(xs, ys) {
  const n = xs.length;
  const lx = xs.map(Math.log), ly = ys.map(Math.log);
  const mx = lx.reduce((a, b) => a + b) / n, my = ly.reduce((a, b) => a + b) / n;
  let sxx = 0, sxy = 0, syy = 0;
  for (let i = 0; i < n; i++) {
    sxx += (lx[i] - mx) ** 2; sxy += (lx[i] - mx) * (ly[i] - my); syy += (ly[i] - my) ** 2;
  }
  const b = sxy / sxx, a = my - b * mx;
  let sse = 0;
  for (let i = 0; i < n; i++) sse += (ly[i] - (a + b * lx[i])) ** 2;
  const df = n - 2;
  const se = Math.sqrt(sse / df / sxx);
  const t = df <= 30 ? T975[df - 1] : 1.96;
  return { b, lnC: a, se, lo: b - t * se, hi: b + t * se, r2: 1 - sse / syy, t, df };
}

function asciiPlot(sizes, series, names, marks) {
  const W = 60, H = 18;
  const xs = sizes.map(Math.log);
  const allY = series.flat().map(Math.log);
  const x0 = Math.min(...xs), x1 = Math.max(...xs), y0 = Math.min(...allY), y1 = Math.max(...allY);
  const grid = Array.from({ length: H }, () => Array(W).fill(" "));
  series.forEach((ys, si) => ys.forEach((y, i) => {
    const cx = Math.round((xs[i] - x0) / (x1 - x0) * (W - 1));
    const cy = H - 1 - Math.round((Math.log(y) - y0) / (y1 - y0) * (H - 1));
    grid[cy][cx] = marks[si];
  }));
  console.log(`ln T (c) від ${y0.toFixed(2)} до ${y1.toFixed(2)}; ln n від ${x0.toFixed(2)} до
    ${x1.toFixed(2)}`);
  grid.forEach(r => console.log("|" + r.join("")));
  console.log("+" + "-".repeat(W));
  console.log(names.map((n, i) => `${marks[i]} = ${n}`).join("   "));
}

const sizes = [50, 75, 100, 150, 200, 250, 300, 400];
const REPEATS = 5;
const rng = makeRng(2026);

const impl = [

  { name: "Наївна, θ(n³)", theory: 3.0, times: [] },
  { name: "Блокова, θ(n³)", theory: 3.0, times: [] },
  { name: "Оптимізоване ядро, θ(n³)", theory: 3.0, times: [] },
];

console.log("Перевірка коректності (макс. |різниця| відносно наївної):");
for (const n of sizes) {
  const A = randomMatrix(n, rng), B = randomMatrix(n, rng);
  const Ar = toRows(A, n), Br = toRows(B, n);
  const ref = mulNaive(Ar, Br, n);
  const c2 = mulBlocked(A, B, n), c3 = mulTuned(A, B, n);
  let d2 = 0, d3 = 0;
  for (let i = 0; i < n; i++) for (let j = 0; j < n; j++) {
    d2 = Math.max(d2, Math.abs(ref[i][j] - c2[i * n + j]));
    d3 = Math.max(d3, Math.abs(ref[i][j] - c3[i * n + j]));
  }
  console.log(`  n=${n}: блокова ${d2.toExponential(1)}, ядро ${d3.toExponential(1)}`);
  if (d2 > 1e-9 || d3 > 1e-9) throw new Error("Реалізації дають різні результати!");

  impl[0].times.push(timeMedian(() => mulNaive(Ar, Br, n), REPEATS));
  impl[1].times.push(timeMedian(() => mulBlocked(A, B, n), REPEATS));
  impl[2].times.push(timeMedian(() => mulTuned(A, B, n), REPEATS));
}

console.log("\nЧас (медіана з " + (REPEATS - 1) + " вимірів, с):");
console.log("n".padEnd(6) + impl.map(x => x.name.padEnd(28)).join(""));
sizes.forEach((n, i) =>
  console.log(String(n).padEnd(6) + impl.map(x => x.times[i].toFixed(5).padEnd(28)).join("")));

console.log("\nОцінка показника степеня b (T = C·n^b), 95 % ДІ (t-розподіл, df = " + (sizes.length - 2) + "):");
for (const im of impl) {
  const f = fitPowerLaw(sizes, im.times);
  const inside = f.lo <= im.theory && im.theory <= f.hi;
  console.log(`${im.name}: b = ${f.b.toFixed(2)}  [${f.lo.toFixed(2)}; ${f.hi.toFixed(2)}]  ` +
    `R² = ${f.r2.toFixed(3)}  теор. = ${im.theory}  -> ` +
    (inside ? "теорія потрапляє в ДІ" : "теорія НЕ потрапляє в ДІ"));
}
```

```
console.log("\nГрафік у логарифмічних осях:");
asciiPlot(sizes, impl.map(x => x.times), impl.map(x => x.name), ["N", "B", "K"]);

console.log("\nПрискорення відносно наївної на найбільшому n=" + sizes[sizes.length - 1] + ": блокова ×" +
  (impl[0].times.at(-1) / impl[1].times.at(-1)).toFixed(1) + ", ядро ×" +
  (impl[0].times.at(-1) / impl[2].times.at(-1)).toFixed(1));
```